

Object Encodings for Dynamically Typed Languages

Dave Mason^{1,*}, Nishil Kapadia^{1,†}

¹Toronto Metropolitan University, Toronto, Canada

Abstract

In dynamically typed programming languages, variables and expressions do not have types associated with them; instead, values have types. This means that selecting the correct machine instructions to operate on data requires run-time dispatch based on the types associated with the values. The simplest encoding strategy is to store every object in a memory heap, where each object is tagged with its type. This approach places significant pressure on both the memory subsystem and the memory allocation system. At the other extreme, the most complex encoding strategies place as many values as possible into immediate representations to reduce this pressure.

This paper describes seven different encoding strategies, ranging from all-memory representations to NaN-based encodings and evaluates their relative performance compared with a baseline implementation that uses no encoding. We evaluate their relative performance on two architectures (AArch64 and x86-64) using recursive Fibonacci benchmarks that stress both integer and floating-point arithmetic paths under inlined and non-inlined dispatch. Our results reveal a fundamental performance trade-off: encoding with minimal tag-width achieves the fastest integer arithmetic but must heap-allocate floats, while NaN-boxing stores doubles without the overhead but penalizes integer extractions. Heap-only (*ptr*) encodings incur order-of-magnitude penalties from cache pressure regardless of architecture. Spur encoding achieves good performance in some cases, but loses in more complex situations. This paper also introduces the Zag encoding, which achieves better overall performance to NaN-boxing across both workloads while offering a wider immediate integer range (55-bit versus 47-bit) and direct class identification without memory lookups.

Keywords

Dynamic Types, Object Encoding, Performance

1. Introduction

For the vast majority of modern von Neumann architecture computers, data (whether stored in memory or registers) is a sequence of bits and the interpretation of those bits is determined by the instructions that are applied to operate on them. So an *integer add* instruction interprets the parameters as 2s complement integers, whereas a *float addition* instruction would interpret the same bits as an IEEE-754 floating-point number.

In statically typed programming languages, the compiler has complete knowledge of the types of operands, and hence the instructions to generate, because every variable and function has a defined type (whether provided explicitly or inferred).

Compilers for dynamically typed languages have no such advance knowledge. While compilers and language systems such as Self[1] and Just-In-Time (JIT) compilers may derive local knowledge of types (using type inference, flow analysis, sampling and other techniques) the ground truth is encoded in the values that are provided and computed. This means that implementers of such languages must choose a mechanism to associate the type with every value.

We have chosen a range of encodings and implemented them as alternatives in a Smalltalk run time system. We have then timed the various encodings in a straightforward implementation of a Fibonacci function evaluated over integer and float domains. While our system is primarily oriented to running Smalltalk, the encodings should have broad applicability.

Modern processors fetch instructions, decode them, and put them in the execution pipeline. This means that unconditional branches and calls to fixed addresses have almost no cost as long as the

IWST-2026: International Workshop on Smalltalk Technology, July 2026 Plovdiv, Bulgaria

*Corresponding author.

†These authors contributed equally.

✉ dmason@torontomu.ca (D. Mason); n1kapadia@torontomu.ca (N. Kapadia)

id 0000-0002-2688-7856 (D. Mason)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

pipeline remains relatively full. Similarly, conditional branches are relatively inexpensive, because of branch prediction hardware and sometimes decoding along more than one potential execution path. What *are* expensive are indirect branches because only after the data is available can the fetch-decode logic start adding to the pipeline. Pipelines can execute instructions out-of-order and sometimes in parallel, as long as dependencies are observed.

There are many options in terms of dynamically-typed language implementation, from byte interpreters to threaded execution to JIT'ed native code. However the encoding choices will affect all of the implementations, to varying degrees.

Paper outline. Section 2 describes the encodings we considered. Section 3 describes ways to encode floating point values. Section 4 describes the experiment and the observed results. Section 5 contains some conclusions and future work.

2. Object Encodings

In this section we describe several different encodings that have been used or proposed in some dynamic language system. For each, we show a possible bit encoding and describe the properties of the encoding.

[2] provides a comprehensive taxonomy of such mechanisms ranging from memory allocation of all objects, to complex tagging of values.

2.1. Encoding Considerations

Encoding values, particularly for modern architectures, have three considerations:

Determining types. Since operations must be dispatched at runtime based on the types of their operands, the type of each value must be determined before an operation can proceed. If the type information is in memory, this can incur a potential pipeline stall while data can be loaded.

Accessing values. In order to perform operations, the values must be made available to the CPU. Accessing values in memory is an order of magnitude slower than accessing them in registers, and even if the values happen to be in cache they will be several times slower than in registers.

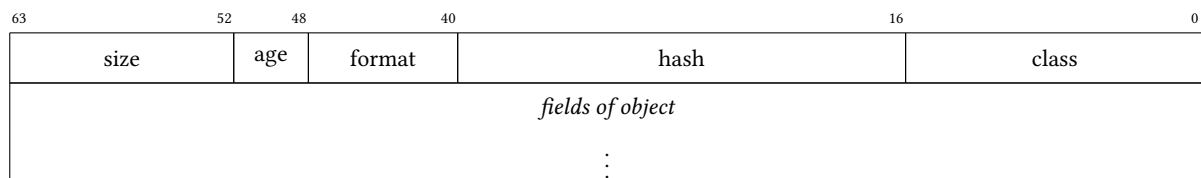
Supporting Memory Management. Dynamically-typed languages invariably have an automatic memory management system, whether reference counting or garbage collection. Objects need to be allocated, and reclaimed when no longer needed. While memory management has improved significantly over the decades, there remains a significant cost to both actions, and the more objects allocated in memory, the greater the cost. We use the term **churn** to refer to the allocation and subsequently collecting in-memory objects.

2.2. Encodings

In an object-oriented language, objects are either mutable or immutable, and this distinction shapes how they can be encoded. Mutable objects must be allocated in memory, and references to them are passed around so that any mutation is visible to all references pointing to the same object. These allocations are typically on a heap and have a header word encoding their class, size, and garbage collection metadata. Immutable objects, by contrast, may be representable without heap allocation.

This paper assumes 64-bit words, 64K classes, and our encoding model throughout. All the objects allocated in these benchmarks go into the per-process nursery heap, which is private to the process and can hence use very cheap allocation and copying collection. Another design would have significantly worse performance when objects must be heap allocated.

For example, our heap object looks like:



The header contains: `size` is the 12-bit number of words the object occupies; `age` is the 4-bit age of the object; `format` is the 8-bit format of the object; `hash` is the 24-bit identity hash; and `class` is the 16-bit class of the object. The header is followed by the fields of the object. The reference to the object is the address of the header word.

There are many possible ways to encode the range of immutable values that arise in normal calculations. In this section we discuss 7 possible encodings that encompass the range from all values being in memory, to treating as many things as immediate as possible. We examine each from the perspective of the three considerations outlined in section 2.1. Table 1 summarizes the important encoding considerations for the encodings discussed below.

Encoding	Determining type & Accessing Value	Memory Considerations
In Memory	memory access	every created immutable requires allocation
Tag SmallInteger	direct for integer; memory access for all others	integers avoid allocation ; all other immutables require allocation
Spur	direct for integer, character, most floats including 0	integer, character, and most float values avoid allocation
SpurNZ	direct for integer, character, most floats except 0	integer, character, and most float values avoid allocation
Zag-Spur	direct for integer, most floats, and all immediates	integer, most floats, and all immediate values (boolean, nil, character, symbol) avoid allocation
NaN Encoding	direct for most immutables	most immutables avoid allocation
Zag	direct for most immutables	most immutables avoid allocation

Table 1
Considerations of Encodings

2.2.1. Every Object in Memory - ptr

This is the simplest model. Every object - mutable or immutable - is encoded as a word-aligned address:



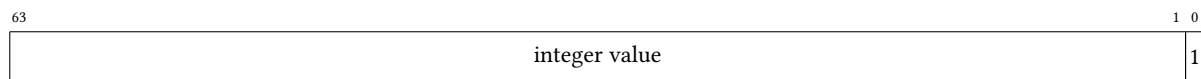
Here, the low 3 bits are zero as part of the word-aligned address, and the high 16 bits are zero due to hardware limitations.

Determining the value type requires a memory access to get the class from the header, as does accessing the actual value. The result of every operation that produces a value must be heap-allocated, resulting in significant allocation pressure; though most values are short-lived, placing pressure on the garbage collector. Inter-operation with foreign functions is complicated, as pointers must be wrapped in heap-allocated objects before use. Immutable values `nil`, `true`, `false`, and ASCII Characters are normally allocated at fixed addresses to gain some performance by allowing hardcoded memory address comparisons and we assume this convention.

2.2.2. Tag *SmallInteger* - taggedInt

Early Lisp[3], Self[4], and Smalltalk[5] implementers observed that the vast majority of runtime values are *SmallInteger* objects. Most implementations tag small integers with a 1-bit tag either in the low bit, keeping all other addresses naturally aligned by objects on at least a word boundary. Less commonly, the sign bit can be used instead. This encoding was significant enough that the SPARC architecture provided tagged arithmetic instructions that checked the low bit of the operands to trap on non-integer values.

Here *SmallInteger* values are encoded as:



and everything else remains encoded as the address:



Note that the low bit distinguishes the integer value from the reference.

A drawback of this encoding is the loss of one bit of precision; however, most such languages automatically promote to arbitrary-precision integers on overflow, and on 64-bit systems the reduced range is rarely a practical concern.

There is also an alternative to this where the integer case uses a 0 low bit and the pointer case uses a 1 low bit. This is slightly faster for *SmallInteger* at the expense of being slightly slower for everything else.

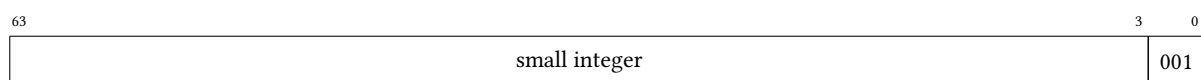
2.2.3. Tag *SmallInteger* , *Character* , *Float* - spur

The Spur[6, 7] encoding extends the 1-bit tag to 3 bits and supports immediate encodings for Unicode characters and a subset of floating-point values in addition to *SmallInteger*.

The default encoding for everything not described below remains encoded as the address:



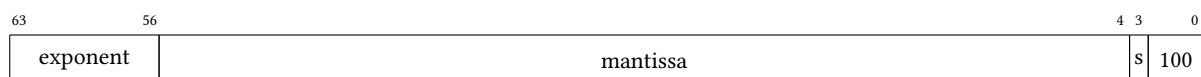
The encoding for *SmallInteger* is:



The encoding for *Character* is:



The encoding for *ImmediateFloat* is:



Examining the low 3 bits of an object, an *and* instruction disambiguate the encodings. Only the pointer tag is used directly; all the others require some decoding before use.

The *ImmediateFloat* encoding preserves the full 52-bit mantissa of IEEE-754 double precision, but reduces the exponent range to $\pm 5.87 \times 10^{-39}$ to $\pm 6.80 \times 10^{38}$. The conversion algorithms are detailed in [8] and require 7–8 instructions. Float values outside this exponent range, as well as *inf* and *nan*, are heap-allocated.

2.2.4. Tag `SmallInteger`, `Character`, `Float` - `spurNZ`

This is identical to `spur`, except the float encoding doesn't try to encode zero as an immediate, and is hence very slightly faster for floating point, and very slightly slower for integer!

2.2.5. Tag `SmallInteger`, other immediate values, `Float` - `zagSpur`

ZagSpur extends Spur by re-purposing the `Character` tag (010) as a general *immediates* tag, allowing any class with a 5-bit index to be encoded as an immediate value. This means `True`, `False`, `UndefinedObject`, `Character`, `Symbol`, and certain block closures that capture no mutable state all avoid heap allocation. It also codes the `nil` object as all zeros - the same as a C null value.

The default encoding for everything not described below remains encoded as the address:

63	48	3	0
0			address
			000

The encoding for `SmallInteger` is:

63	3	0
small integer		001

The encoding for immediate classes (including `Symbol`, `True`, `False`, `UndefinedObject`, `Character`, and some kinds of block closures) is:

63	8	3	0
contents			class
			010

where the `class` encodes up to 32 special classes in exactly the same way as Section 2.2.7.

The encoding for `ImmediateFloat` uses exactly the same encoding as `spur`(2.2.3):

63	56	4	3	0
exponent		mantissa		s
				100

Only the pointer is used directly; all the others require some decoding before use.

Values whose class index exceeds 5 bits, and floats outside the encodable exponent range, are heap-allocated.

2.2.6. NaN Encoding - `nan`

The IEEE-754 64-bit floating point number is formatted as:

63	52	0
s	exponent	matissa

where if all the exponent bits are 1, the value is determined to be not-a-number (NaN). NaN encoding utilizes the large number of code points in the encoding that do not represent valid floating-point numbers, by using those bit patterns to represent values of other types, including pointers and `SmallInteger`. A version of this encoding has been used by Spidermonkey[9], and was the originally planned encoding for our Smalltalk. There is also a US patent for this[10] that expires in 2028. There are various ways one might encode this, but for the purposes of this experiment we use the following. `SmallInteger` is encoded as:

63	50	0
11111111111111		small integer

To check for a `SmallInteger` we do an unsigned comparison, anything with the high 14 bits equal to 1 is a small integer, and the low 50 bits are a 50-bit two's-complement integer value. 51 bits should be available, but the x86-64 hardware-generated NaN value interferes.

The floating point numbers are treated as natural values, and only one value is designated as NaN - the value generated by the hardware. We recognize these by testing for NaN. If it's not a NaN then it's a floating point value. There is one false negative, so we check for that on a low-frequency path.

The rest of the values are:



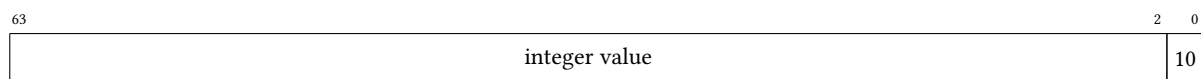
where `tag` is a 4-bit tag: 0xF encodes heap references; tags 1–9 encode immediate `BlockClosure` subclasses with varying parameter types; tags A–D encodes other immediate classes (symbols, characters, etc.).

This encoding recognizes all `SmallInteger` values with a single comparison and `Float` values with an additional comparison and floats require no conversion before use. All the other values, including references, require more bit manipulation to extract the class and value. Immediate values carry up to 48 bits of payload. As mentioned we can also encode some of the same `BlockClosure` subclasses values as described in Section 2.2.7, but with much-restricted parameters due to reduced available bit-width.

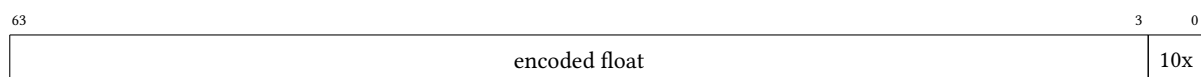
2.2.7. Tag almost all immutable values - zag

We actually ran almost 20 different encodings, trying low tags, high tags and mixed tags. The best overall encoding we found we call Zag encoding. It uses mixed encoding - some low bits and some high bits.

The first format is `SmallInteger` with a value of 2 in the bottom 2 bits, which can be detected with an and of 2 being nonzero:



Second is `Float` which is a variant of FST2 (see Section 3) with a value of 4 or 5 in the low 3 bits which can be detected with an and of 4 being nonzero:



Decode is to subtract 5 from the encoded values (as u64) and then to rotate right 5, and cast to float. Encode is to cast as u64, rotate left 5, and then add 5. If the result anded with 6 equals 4, then the encoding is successful.

If it isn't either of the above, then the interpretation is:



where the class is one of 32 possible compact classes, including native, heap, True, False, UndefinedObject, Symbol, Character, and several kinds of block closures. If the class is heap, then the low 48 bits are a pointer. The extra bits are mostly used in the block closures to encode things like instance variable indices, or small constants. If the class is native, then this is a C/C++/Zig pointer, including NULL (nil).

3. Float Encoding

An important component of this work is to incorporate floating point numbers as immediate values encoded in an "Object". In this section we explore the cost of encoding/decoding using various schema, as well as the usage-based statistical percentage of coverage of each encoding. It is conventional wisdom that *SmallInteger* is very heavily used, which justifies special handling for those values. The use of *Float* is less well understood. For statically-typed languages, there is a study[11] of code on GitHub that found that over 62% of the extensive code they sampled used floating-point. We infer that it is likely to be increasingly important in dynamically-typed languages, although past, poor, implementations probably have reduced the importance relative to statically-typed languages.

Contemporaneous with this research, the "Float Self-Tagging"[12] paper was being created. That paper discusses a range of encodings for floating point values that support including immediate floating point values as objects. This paper expands upon that by incorporating those (and similar) encodings with encoding of other immediate objects.

As part of the RISC-V floating point design an extensive analysis was done of how floating point values cover the possible range of bit patterns[13]. They published the data from an analysis of 72 benchmarks including the SPEC floating point benchmarks. While some weighting of those raw results might be appropriate, we had no basis to assign such a weighting, so treated all of the data equally. Table 2 shows the range of exponent values and how the encodings described below cover them. The

Exponent	Range	Frequency	spur	spurNZ	fst1	fst2	fst4	zag6	NaN
00000000000	0	0.04%	✓	✗	✓	✓	✓	✓	✓
00000	$\dots 2.05 \times 10^{-289}$	0.06%	✗	✗	✓	✓	✓	✓	✓
00001	$\dots 3.79 \times 10^{-270}$	1.17%	✗	✗	✗	✓	✓	✓	✓
0001	$\dots 1.29 \times 10^{-231}$	1.19%	✗	✗	✗	✗	✓	✓	✓
001-010	$\dots 1.73 \times 10^{-77}$	0.89%	✗	✗	✗	✗	✗	✓	✓
0110	$\dots 5.88 \times 10^{-39}$	0.69%	✗	✗	✗	✗	✓	✓	✓
01110	$\dots 1.08 \times 10^{-19}$	1.42%	✓	✓	✗	✓	✓	✓	✓
01111-10000	$\dots 3.69 \times 10^{19}$	94.51%	✓	✓	✓	✓	✓	✓	✓
10001	$\dots 6.81 \times 10^{38}$	0.01%	✓	✓	✗	✓	✓	✓	✓
1001	$\dots 2.32 \times 10^{77}$	0.01%	✗	✗	✗	✗	✓	✓	✓
101-110	$\dots 3.11 \times 10^{231}$	0.00%	✗	✗	✗	✗	✗	✗	✓
1110	$\dots 1.06 \times 10^{270}$	0.00%	✗	✗	✗	✗	✓	✓	✓
11110	$\dots 1.95 \times 10^{289}$	0.00%	✗	✗	✗	✓	✓	✓	✓
11111	$\dots nan$	0.04%	✗	✗	✓	✓	✓	✓	✓

Table 2
Float encoding coverage

premise is that if a value is not encodable (i.e. a ✗), it will be allocated in memory, with the extra memory accesses implicit in that.

Encoding	Encode	Unencodable	Decode	Coverage	Tags Used
spur	0.851ns	0.989ns	0.609ns	95.98%	4 (5,6,7 reserved)
spurNZ	0.622ns	0.619ns	0.619ns	95.94%	4 (5,6,7 reserved)
fst1	0.595ns	0.586ns	0.482ns	94.61%	4 (5,6,7 reserved)
fst2	0.593ns	0.585ns	0.497ns	97.21%	4,5 (6,7 reserved)
fst4	0.711ns	0.640ns	0.497ns	99.11%	2,3,6,7
zag6	0.489ns	0.450ns	0.488ns	99.99%	2,3,4,5,6,7
nan	0.396ns	0.000ns	0.457ns	100.00%	N/A

Table 3
Float encode/decode times (aarch64)

Table 3 shows timing for implementations of encoding and decoding of each of the encodings. The "Encode" column is the time to encode values that are valid for the particular encoding. The

“Unencodable” column is the time to attempt encode values that are not valid for the particular encoding. The “Decode” column is the time to recognize and decode validly encoded values. “Coverage” is the sum of the frequency information in table 2 for the check marks for this particular encoding. “Tags Used” says what range of tag bits (mostly low bits) are used as to signify an floating-point value. Reserved indicates false-positives assuming the bottom 3 bits are used for tags, requiring that if those tags are used they must be tested for first, or that the testing code would have to be made more explicit.

Coverage

All of the encodings except NaN have incomplete coverage of floating-point values. If a value isn’t encodable then it will have to be allocated in memory. As mentioned before, the coverage numbers above use unweighted data, so may over-represent a lot of ranges that are actually extremely rare. In particular, the few percent of very small values probably come from matrix inversions. So the Spur and FST1 type encodings are probably pretty complete, especially if you use statically allocated values for the zero, infinity and generated-NaN values for Spur and SpurNZ.

3.1. Spur

Spur encoding was developed for the OpenSmalltalk project, described in detail in [8, 6]. In that system, the low 3 bits are used as tags with 4 being used for floating-point, so the fact that 5,6 and 7 would be false-positives is irrelevant. By treating 0 as a special case, it is able to cover quite a broad range of values: $5.88 \times 10^{-39} \dots 6.81 \times 10^{38}$.

SpurNZ avoids encoding the 0 case and thereby can be slightly faster.

3.2. Float Self Tagging

The various Fst encodings discussed in [12] use from 1 to 4 tag values to encode a limited set of floating point values.

3.3. Zag6

Zag6 is a novel encoding that uses 6 tag values to encode virtually all of the most-used floating point values.

3.4. NaN

NaN encoding treats floating point values naturally and fills in other values by using the 2^{53} bit patterns in the IEEE floating point standard that encode for Not-A-Number.

Implementations

Implementations of all of these float encodings are available at <https://github.com/Zag-Research/Zag-Smalltalk/blob/main/zig/zag/encoding/floatEncoding.zig>

4. Experimental Design and Results

4.1. Experimental Design

Our Smalltalk was originally going to use NaN-encoding, and even after the decision was made to use the Zag-encoding described in section 2.2.7, the code for both was carried in parallel, although bit-rot set in for the NaN encoding as additional features were implemented. However the question of the best encoding has always remained.

There is a single enumeration in the runtime configuration module that determines which encoding will be used, so we can easily compile with any one of these encodings.

The experiment is to run the Fibonacci algorithms shown in Listing 1.

Listing 1: fibonacci

```
fibonacci1
    self <= 2 if True: [ ^ self ].
    ^ (self - 1) fibonacci + (self - 2) fibonacci
```

It will be run twice with *SmallInteger* and *Float* as receiver. We evaluate 40 *fibonacci* or 40.0 *fibonacci*. This will exercise the integer and floating-point paths. Note that for this particular benchmark, all of the floating point calculations are representable in all of the limited-range floating point encodings.

The experiment is run on two different architectures:

AARCH64 This is an Apple M1 Firestorm core running at 2.99GHz with a per-core caches: L1 192KB instruction and 128KB data. There is also a 12MB L2 cache shared by all performance cores, but since this is the only process running on the performance cores, this is effectively available to the benchmark. Finally there is a 8MB System Level Cache. The benchmark is fixed to a single core and address-space randomization is off. The core’s scaling_governor has been set to performance.

X86-64 This is an Intel® Core™ i7-8700 CPU running at 3.20GHz with per-core caches: L1 32KB instruction and 32KB data, and L2: 256KB. There is also a 12MB cache shared by all cores. The benchmark is fixed to a single core and address-space randomization is off. The core has been removed from the operating system’s pool of assignable cores, so the caches will remain undisturbed.

4.2. Results

Table 4 shows the total execution time of the various encodings to execute the benchmark (40 *fibonacci*) for integer and floating-point values.

Encoding	Integer	Float
nan	3136ms	2959ms
onlyFloat	N/A	2580ms
onlyInt	2601ms	N/A
ptr	4712ms	5075ms
spur	2667ms	3497ms
spurNZ	2746ms	3412ms
taggedInt	2666ms	4844ms
zag	2705ms	2940ms
zagSpur	2678ms	3067ms

AArch64 (Apple M1)

Encoding	Integer	Float
nan	4694ms	4601ms
onlyFloat	N/A	4124ms
onlyInt	4268ms	N/A
ptr	7209ms	7612ms
spur	4580ms	5386ms
spurNZ	4585ms	5350ms
taggedInt	4408ms	7959ms
zag	4380ms	5183ms
zagSpur	4409ms	5402ms

x86-64 (Intel i7-8700)

Table 4

Total timing for encodings for 40 *fibonacci*

The *onlyInt* and *onlyFloat* use no encoding and no type-checking; everything is assumed to work correctly, so these are the fastest our model could possibly execute.¹ As can be seen, there is significant overhead in our execution model that is used in the non-native executions.

All timings are the median of 10 runs following 3 runs of warmups. They are run on Linux systems running in console mode. We calculate the mean, median and geometric mean for each example and they are essentially equal in all cases. They are also essentially equal from run to run. This is important to realize because some of the results are surprising - two different encodings that do some of the same instructions can have significantly different timings.

¹We are working on faster execution models, but that is irrelevant to this experiment.

The AArch64 architecture seems to be better tuned for these encodings than x86-64 as all of the timings are faster - often significantly so, despite the x86-64 machine being about 20% faster based on a native fibonacci implementation.

4.2.1. Integer Performance (Integer)

On AArch64, `taggedInt` dominates integer performance, followed by - `zagSpur` and `spur`. this ordering - although highly repeatable - is a function of the micro-architecture as they all check to see if a bit is set. they only differ in cases that are not exhibited in this benchmark, but they have slightly different execution times due to slightly different ordering of instructions because `zagSpur` handles more possible immediate values.

Similar results are visible on x86-64, where `taggedInt` is beaten out by `zag` and `zagSpur`, and the `spur` is a bit slower. `nan` is substantially slower because *NaN-boxing* requires a bit-rotation and masking to recover the 50-bit integer payloads and x86-64 appears to penalize these operations more heavily than M1.

Not surprisingly, `ptr` encoding is slowest for integers for both architectures, roughly an order of magnitude slower than `taggedInt`. Every integer operation requires a heap reference to read the value, polluting the cache with millions of small boxed objects. Then the objects need to be garbage collected.

4.2.2. Float Performance

While all the values used in the test are encodable by `Spur` and all subsequent encodings, they take different times to recognize and encode/decode. `nan` encoding achieves the best float performance on x86, but is slightly slower than `zag` on AArch. This is the fundamental advantage of *NaN-boxing*: IEEE 754 doubles are stored *unboxed* as the raw 64-bit representation, and type checking reduces to a simple *NaN* comparison with no memory indirection required for float operands. `nan` still wins, even on AArch, because all possible floating-point values are encoded, whereas `zag` only captures 97.2% of values as immediate values.

Float performance is thus dominated by where the encoding supports immediate floating-point values. Encodings that force heap allocation for floats pay 3-6x penalty relative to `nan` encoding.

4.2.3. Pareto Analysis

The Pareto frontier contains the points that represent optimal trade-offs between two parameters. In this case, we are looking at integer versus floating-point performance in figure 1. The Pareto frontiers don't intersect, showing that the AArch64 dominates the x86-64 in this experiment. Note that this plot shows the total execution time for each encoding, but the origin is not shown so the visual scale can be deceptive.

For AArch64, many of the encodings are on the frontier:

- `zag` - fastest float, very slightly slower integer;
- `zagSpur` - slightly slower float cost, slightly faster integer;
- `spur` - a fair bit slower floating point, marginally faster integer;
- `taggedInt` - best integer performance, worst float performance on the frontier.

Encodings such as `ptr` are Pareto-dominated as they are outperformed in *both* dimensions by other encodings and can be ruled out for any modern production general-purpose workload. Their value lies primarily as experimental baselines that isolate the cost of heap-only object representations.

The `zag` encoding occupies the most balanced position on the Pareto frontier, delivering competitive performance on both integer and float benchmarks.

For x86-64, only 2 encodings are on the frontier:

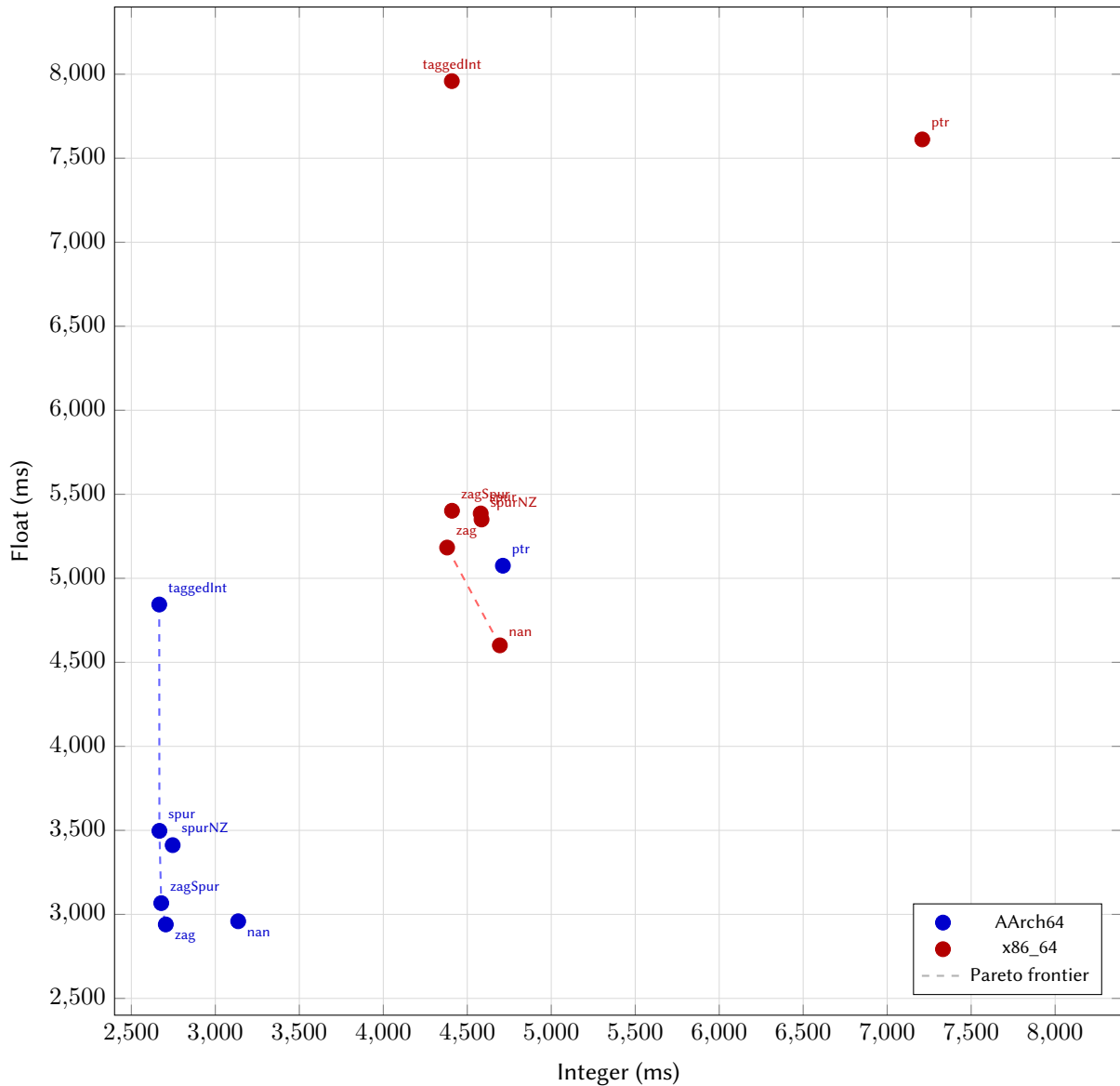


Figure 1: IntegerBr vs Float

- `nan` - best float performance, moderate integer cost;
- `zag` - only slightly slower float performance, but better integer, the most balanced position on the frontier;

5. Conclusions

There are few surprises coming out of this experiment: allocating objects in memory is slow, so encodings that need to allocate often are going to be slow.

Less obvious findings are that Nan-encoding is very good on x86-64, but a little slow on the integer workload on AArch64. The `zag` encoding has a good balance on x86-64 and is the clear favourite on AArch64. There is no reason to use `spur` as `zagSpur` is a proper superset; they are very slightly faster than `zag` for integer on AArch64 but slower on x86-64, and slower for floating-point on both platforms. Floating-point values are naturally represented in `nan`, which could be useful for some kinds of foreign function work or GPU interaction. Pointers are naturally represented in `zag`, `zagSpur`, and to a lesser extent `spur`, and this could be useful for foreign function interfacing.

All the encodings are available on GitHub at <https://github.com/Zag-Research/Zag-Smalltalk/tree/main/zig/zag/encoding>

Future Work

The Fibonacci benchmark, while exercising core dispatch and arithmetic paths, is a micro benchmark. Real Smalltalk applications exercise object allocation, polymorphic dispatch, string manipulation, and collection operations that may shift the performance balance between encodings.

In particular, encodings with wider tag spaces such as `zag` with its three-bit tag and 16-bit class field may gain advantages in polymorphic dispatch that Fibonacci does not exercise.

The experiment should be extended with macro-benchmarks and application-level workloads to validate whether the rankings observed here hold under realistic conditions. In particular we look forward to validating our hypothesis that the extended range of immediate values that can be encoded in `zag`, `nan`, and `zagSpur` will have significant performance advantages over `spur` and `taggedInt` in more general workloads.

References

- [1] Self Language Project, Self: Prototype-based programming language and environment, 2024. URL: <https://selflanguage.org/>, official website and documentation for the Self language.
- [2] D. A. Gudeman, Representing type information in dynamically typed languages, University of Arizona Technical Report (1993). URL: <http://jimmyhmler-bucket.s3.amazonaws.com/pdfs/programming-languages/6586051a27daa629dd94e06306cb3edad4f62dcf9579f65358e7940d8fed7ec0.pdf>.
- [3] G. L. Steele, Jr., Data Representations in PDP-10 MacLISP, Technical Report AI Memo 420, Massachusetts Institute of Technology, Artificial Intelligence Laboratory, 1977. URL: <https://dspace.mit.edu/handle/1721.1/6278>.
- [4] D. Ungar, R. B. Smith, Self: The power of simplicity, in: Conference proceedings on Object-oriented programming systems, languages and applications, 1987, pp. 227–242.
- [5] L. P. Deutsch, A. M. Schiffman, Efficient implementation of the smalltalk-80 system, in: Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, POPL '84, Association for Computing Machinery, New York, NY, USA, 1984, p. 297–302. URL: <https://doi.org/10.1145/800017.800542>. doi:10.1145/800017.800542.
- [6] E. Miranda, C. Béra, A partial read barrier for efficient support of live object-oriented programming, in: Proceedings of the 2015 International Symposium on Memory Management (ISMM), 2015, pp. 93–104. doi:10.1145/2754169.2754186.
- [7] E. Miranda, A spur gear for cog, ??? URL: <http://www.mirandabanda.org/cogblog/2013/09/05/a-spur-gear-for-cog/>.
- [8] C. Béra, 64 bits immediate floats, <https://clementbera.wordpress.com/2018/11/09/64-bits-immediate-floats/>, 2018. Blog post on the representation of immediate floating point numbers in the 64-bit Spur/OpenSmalltalk VM.
- [9] Mozilla, Spidermonkey javascript engine, 2025. URL: <https://spidermonkey.dev/>, [Online; accessed 2025-04-08].
- [10] J. F. Dahms, D. P. Yach, Floating point encoding systems and methods, 2011. US Patent 7,877,431.
- [11] A. Gilot, T. Wrigstad, E. Darulova, Floating-point usage on github: A large-scale study of statically typed languages, Proc. ACM Program. Lang. 10 (2026). URL: <https://doi.org/10.1145/3798203>. doi:10.1145/3798203.
- [12] O. Melançon, M. Serrano, M. Feeley, Float self-tagging, Proceedings of the ACM on Programming Languages 9 (2025) 1620–1646. URL: <http://dx.doi.org/10.1145/3763108>. doi:10.1145/3763108.
- [13] N. Zurstrassen, L. M. Reimann, N. Bosbach, L. Juenger, R. Leupers, Evaluation of the risc-v floating point extensions, in: DVCon Europe 2023; Design and Verification Conference and Exhibition Europe, 2023, pp. 57–64.